

奥赛罗棋智能模块的算法设计

吴华蕾

(厦门教育学院, 福建 厦门 361004)

【摘要】奥赛罗棋(Othello)是一种常见的双 agent 博弈游戏。本文主要介绍了奥赛罗棋的计算机博弈游戏中智能模块的设计方法, 包括博弈程序设计中采用的搜索方法和对棋局进行估价等的算法设计。

【关键词】博弈搜索、Alpha - Beta 剪枝、估价函数

【中图分类号】TP 311.11 【文献标识码】A 【文章编号】1673 - 0275 (2005) 02 - 0082 - 03

奥赛罗棋是一种博弈游戏, 由两个 agent (称为博弈者) 轮流移动, 直到其中一方获胜 (另一方因此失败), 或双方和局。每个博弈者完全熟悉环境及自己和对方可能的移动方式和影响。许多博弈, 包括国际象棋、西洋跳棋(draughts) 和围棋, 都属于这种类型。

奥赛罗棋棋盘是一个有 8×8 方格的棋盘, 棋子双面: 一面白色、一面黑色。棋子应下在棋盘空格中。开盘时, 棋盘上正中共有 4 粒棋子, 两白两黑呈对角线排列, 由执黑者先行。双方都必须将棋子下在能在直线或斜线方向夹住对方的棋子的地方。被对方棋子夹住的棋子, 会翻转变成对方的棋子。只有在可以翻转棋子的地方才可以下子。如果棋盘上没有地方可以下子, 则对方连下。双方都没有棋子可下时棋局结束, 棋盘上棋子多的一方获胜。双方棋子数目相同, 则和棋。

博弈通常都用图来描述自己的状态空间, 直观上我们可以采用 8×8 数组来记录黑白博弈者在 8×8 网格中的不同位置。这种方法的缺点是程序运行时速度会受影响, 因为需用二重循环来遍历棋盘。所以也可采用一维数组定义棋盘。

用算子表示博弈的移动, 算子把一种状态描述转换为另一种描述, 由一个开始节点和每个博弈者的算子隐式定义博弈图, 建立搜索树。

如同大多数博弈一样, 奥赛罗棋可采用博弈搜索和 Alpha - Beta 剪枝技术。

命名对弈双方为 MAX 和 MIN。程序的任务是为 MAX 寻找最佳的移动。假设 MAX 先移动, 接着两个博弈者轮流移动。因此, 博弈搜索树中深度为偶数的节点, 对应于 MAX 下一步移动的位置, 称为 MAX 节点; 深度为奇数的节点对应于 MIN 下一步移动的位置, 称为 MIN 节点 (博弈树的顶节点深度为 0)。k 层包括深度为 $2k$ 和 $2k + 1$ 的节点。通常用层数给出博弈树的搜索程度, 它可以表示出向前预测的 MAX 和 MIN 交替运动的回合数。

完全的搜索对于大多数博弈来说是不可行的, 所以应该使用有限范围搜索方法。人为地设定几个终止条件,

如: 时间限制、存储空间限制或在搜索树中最深节点的深度。

搜索结束后, 需从搜索树中选取一个最佳首次移动, 这个选取方法可以对搜索树的叶节点采用静态估价函数。此估价函数衡量每一个叶节点位置的“值”。通常分析博弈树时, 对 MAX 有利的位置, 估价函数将赋予正值; 对 MIN 有利的位置赋予负值, 接近零的值表示该位置对 MAX 和 MIN 都一样。

一个最佳的走步可以由一个最小最大化过程产生。假设轮到 MAX 从搜索树的叶节点中选取, 他肯定选择拥有最大值的节点。因此, MIN 叶节点的一个 MAX 双亲节点的倒推值就等于叶节点的静态估价值中的最大值。另一方面, MIN 从叶节点中选取时, 必然会选值最小的节点 (即最负的值)。既然如此, MAX 叶节点的 MIN 双亲节点被分配一个倒推值, 它等于叶节点静态估价值的最小值。在所有叶节点的父节点被赋予倒推值后, 开始倒推另一层, 假定 MAX 将选择有最大倒推值的 MIN 后继节点, 而 MIN 会选择有最小倒推值的 MAX 后继节点。

我们继续逐层对节点估价, 直到最后开始节点的后继者被赋予倒推值。假定 MAX 首先移动, MAX 将选择有最大倒推值的节点作为它的走步。

整个过程的有效性基于这样的假设: 开始节点的后继的倒推值比直接从静态估价函数中得到的值更可靠。由于倒推值基于在博弈树中的预先推算, 并且取决于在博弈结束时发生的一些特性, 这些值往往更加切合实际。

但一个棋局的搜索树往往十分庞大, 要将它完全产生, 势必浪费大量的系统时间及空间, 而将叶节点的估价、计算倒推值与树的产生同时进行, 就可能大量减少所需搜索的数目。所以, 在搜索过程中加入 Alpha - Beta 剪枝可以跳过一些不必要的搜索、估值计算。

采用深度优先搜索, 当节点产生, 便立刻被静态估价, 并假定一旦一个位置可给予倒推值时, 这个值便立刻被计算出来。通过记录倒推值的界限可减少搜索。通常, 当一个节点的后继节点有了倒推值之后, 倒推值的界限就

会被修改，修改原则为：

1) MAX 节点的 α 值（包括开始节点）决不会减小。

2) MIN 节点的 β 值决不会增加。

α 值和 β 值的计算方法：

1) MAX 节点的 α 值等于它的后继节点中的最大最终倒推值。

2) MIN 节点的 β 值等于它的后继节点中的最小最终倒推值。

Alpha - Beta 搜索规则（即剪枝规则）：

1) 当任何 MIN 节点的 β 值小于等于它的 MAX 的祖先节点的 α 值时，则可中止该 MIN 节点以下的搜索。该 MIN 节点的最终倒推值即为它的 β 值。

2) 当任何 MAX 节点的 α 值大于等于它的 MIN 的祖先节点的 β 值时，则可中止该 MAX 节点以下的搜索，该 MAX 节点的最终倒推值等于它的 α 值。

当按规则 1 停止搜索时，称：产生 α 剪枝；当按规则 2 停止搜索时，称：产生 β 剪枝。

经过剪枝，搜索树的叶节点中只有较少的节点需要进行评估，搜索效率大大提高了。

以下为奥赛罗棋游戏程序中加入 Alpha - Beta 剪枝的搜索算法的核心：

```
Double AlphaBeta (int depth ;Double Alpha, Beta ; TPosition
Position)
```

```
{ //定根节点深度 depth = depthmax 叶子节点深度 depth = 0 ;
```

```
if ((depthmax - depth) % 2 == 0) //电脑下棋的节点
```

```
value = - MaxInt ;
```

```
//节点赋初值,初始化为一个不可能达到的最小值
```

```
else //人下棋的节点
```

```
value = MaxInt ;
```

```
//节点赋初值,初始化为一个不可能达到的最大值
```

```
if (depth == depthmax) max = - MaxInt ;
```

```
if (depth == 0)
```

```
{ AlphaBeta = 估值函数的估价 ;
```

```
exit() ;}
```

```
While (每一个合法的可下棋的位置)
```

```
{ 保存棋局 ;
```

```
下棋 ;
```

```
//递归调用,双方返回值互为相反数,不用判断取最
```

```
//大还是最小,更加简洁。
```

```
v = - AlphaBeta (depth - 1, - Beta, - Alpha, Position) ;
```

```
//选择节点中最大或是最小的值
```

```
if (v > Alpha)
```

```
if (v >= Beta)
```

```
{ Result = v ;
```

```
恢复棋局 ;
```

```
Exit() ;}
```

```
else
```

```
Alpha = v ;
```

```
//如果值大于根节点值则赋值
```

```
if (depth == depthmax) and (value > max)
```

```
{ max = value ;
```

```
max_x = i ; //记录 x 坐标
```

```
max_y = j ;} //记录 y 坐标
```

```
}
```

```
恢复棋局 ;
```

```
AlphaBeta = Alpha ;
```

```
}
```

有了高效正确的算法还不足够，对局面的评估是决定棋力的重要因素。要想编写好的估价函数为棋局估价，应该先熟悉奥赛罗棋的战术技巧。

虽然奥赛罗棋游戏中规定对局结束时子多的一方为胜方，但在下棋时并不是每一步能吃越多的子越好，在棋局中应考虑战术，例如：

1、位于棋盘 4 角的棋子是不可能被对方夹吃的，所以，对局时应伺机占据角部的棋位，如果能占据 4 个角部棋位中的 3 个甚至 4 个，常能取胜。因此在程序中，角部棋位的位置价值应该设为一个较高的值。

2、由角部数起，位于第 2 行第 2 列的棋位代号为二二（角部棋位代号为——），即对角线上与角部棋位斜向相邻的棋位。由“战术技巧”第（1）条可知，若布子于二二，易被对方占据——，对己方不利。因此，不宜布子于二二，程序中二二棋位的位置价值应设为不利于己方的负值。

3、布子于边上对方 2 子之间，同时在其他地方向上夹吃对方棋子，称为嵌入。边上嵌入的棋子不但自身安全，而且能在以后的战斗中发挥重要作用。因此，可加入对嵌入子的动态判断。

4、对行动力的判断也很重要。下棋时，如果有很多位置可选择，我们称之为行动力强，反之，我们称为行动力弱。下棋过程中，必须尽量削减对手的行动力，同时增加自己的行动力。而一旦无子可布时，对方会尽力增加连续布子的次数，对己方不利。所以，行动力也需考虑在估值函数中。

一般程序的估值基于许多的参数，如：行动力、潜在行动力、余裕手、边角判断、稳定子。奥赛罗棋游戏程序的估价方法应包括基于棋子的位置价值的估价和基于行动力的估价。

以下为一个奥赛罗棋游戏程序中将棋位价值估算和行动力估值合并计算价值的算法例子：

```
float countvalue()
```

```
{ int i,j ;
```

```
int manMobility, compMobility ; //双方行动力
```

```
int manValue, compValue ; //双方棋位价值
```

```
manValue = 0 ;
```

```
compValue = 0 ;
```

```
if (last == true) //若为终局搜索
```

```
{ for (i = 1 ; i <= 8 ; i++)
```

```
for (j = 1 ; j <= 8 ; j++)
```

```
{ if (ichess[i,j] == compcolor) compValue++ ;
```

```
//电脑的棋子数
```

```
        if (ichess[i,j] == mancolor)    manValue + + ;
//对手的棋子数
    }
    countvalue = compValue - manValue ;
    exit() ;
}
changecost() ;
//根据形势,动态修正棋位价值 cost 数组
for (i = 1 ; i <= 8 ; i + + )
    for(i = 1 ; i <= 8 ; i + + )
        { if (ichess[i,j] == compcolor)
            compValue = compValue + cost[i,j] ;
//电脑的棋位价值
            if (ichess[i,j] == mancolor)
                manValue = manValue + cost[i,j] ;
//对手的棋位价值
        }
    manMobility = 0 ;
    compMobility = 0 ;

    for (i = 1 ; i <= 8 ; i + + )
        for (j = 1 ; j <= 8 ; j + + )
            { if chessable(i,j,compcolor)
                //电脑在位置[i,j]可以下棋
                compMobility + + ;
            //增加电脑的行动力
                if chessable(i,j,mancolor)
            //对手在位置[i,j]可以下棋
                manMobility + + ;
            //增加对手的行动力
            }
    countvalue = m * ( compMobility - manMobility ) + n *
    (compValue - manValue) ;    //估值的线性合并
}
```

正确地使用博弈搜索、Alpha – Beta 剪枝和估价函数技术,就能将奥赛罗棋游戏中的智能模块做好,而系统棋力的增强和程序效率的提高则需要对算法进行不断的充实完善。

(上接第 81 页)

人的公开招聘。采取条件公开、职位公开,目标公开、待遇公开的办法,公开选拔高校急需的专业技术人才,为优秀人才脱颖而出开辟“快车道”。积极鼓励海外留学人员通过兼职、开展合作研究、讲学、进行学术交流等适当形式,为高校的改革发展服务。

4、构建科学的人才培育机制,增强高校人才队伍后劲和潜力

人才培养是人才队伍建设的基础环节,人才培育机制是人才聚集的强劲磁场。高校要发展,人才培养要先行。根据不同类型、不同层次人才的实际需求,科学地确定不同的培养目标、培训内容,拓宽培训渠道,创新培训方式,逐步形成一套较为完整的、覆盖高校人才队伍的分类培养机制。

坚持重点岗位人才重点培养,优秀人才优先培养,紧缺人才抓紧培养的原则,采取切实措施,强化人才教育培训工作,将科学的人才培训机制与终身教育、终身学习结合起来,与理论提高、实践锻炼相结合,在促进高校整体人才素质的提高与升华的同时,增强高校人才队伍的后劲和潜力。

[参考文献]

[1] 李金松,张来民主编:中国人才队伍建设百题[Z]. 中国发展出版社,2003.
[2] 丁振平. 论人力资源管理中的激励机制[J]. 佳木斯大学社会科学学报,2004(2).
[3] 郑小波. 在全球化视野中构筑高校人才队伍高地[J]. 国家教育行政学院学报,2004(2).